

6기 파이썬 프로그래밍 1주차 요약본

1. 파이썬 이해 및 개발환경 설정

1.1 파이썬 이해 및 설치

- **특징:** 1991년에 개발된 인터프리터 프로그래밍 언어로, 간단한 문법과 다양한 라이브러리 제공.
- **장점:** 무료 소프트웨어(오픈소스), 객체지향적, 데이터 분석 및 인공지능 분야에서 널리 사용.
- **설치 방법:** 공식 사이트(<https://www.python.org/downloads>)에서 운영체제에 맞는 설치 파일 다운로드 후 설치.
- **개발환경 옵션:** IDLE(기본 제공), VSCode, PyCharm, Colab 등.

1.2 VSCode 설치 및 기본 사용법

- **특징:** 코드 자동 완성, 디버깅, 터미널 실행 등 다양한 기능 제공.
- **설정 방법:**
 1. VSCode 설치 → Extensions → **Python** 검색 후 설치.
 2. 설치 완료 후 VSCode 종료 후 재시작.
 3. **Ctrl + ~** 로 터미널 열기 → .py 파일 실행 확인

1.3 Colab 기본 사용법

- **특징:** 웹 기반 개발 환경으로 파이썬 설치 없이 코드 실행 가능.
- **활용:** 데이터 분석 라이브러리 기본 설치 및 GPU 사용 가능.
- **실행 관련 단축키:**
 - **Ctrl + Enter** : 셀 실행.
 - **Shift + Enter** : 셀 실행 후 다음 셀로 이동.
 - **Alt + Enter** : 셀 실행 후 새로운 코드 셀 추가.
- **셀 삽입/삭제 관련 단축키**

- `Ctrl + M A` = 코드 셀 위에 삽입
- `Ctrl + M B` = 코드 셀 아래 삽입
- `Ctrl + M D` = 셀 지우기
- `Ctrl + M Y` = 코드 셀로 변경
- `Ctrl + M M` = 마크다운(텍스트) 셀로 변경

*** 단축키가 적용되지 않을 경우 microsoft 키보드 입력기로 변환 필요.

2. 프로그래밍 개념 및 파이썬 기본 요소

2.1 프로그램, 데이터

- **프로그램**: 데이터를 입력받아 처리하고 결과를 출력하는 명령어들의 집합.
- **데이터 타입**: 정수형, 실수형, 문자열, 리스트, 튜플, 딕셔너리 등.

2.2 변수와 명령어

- **변수**: 데이터를 저장하는 메모리 공간의 이름.
- **변수명 설정**: 의미없는 변수명 사용은 지양.
 - 문자, 숫자, 언더바 사용 가능
 - 대문자, 소문자 구분
 - 공백 사용 불가
 - 숫자로 시작 불가
 - 파이썬 예약어 사용 불가

```
False  class  return  is      finally
None   if     for     lambda continue
True   def    from   while  nonlocal
and    del   global not    with
as     elif  try    or     yield
assert else  import pass
break  except in    raise
```

- **명령어**: 데이터를 처리하거나 입출력하는 코드.

```
num = 10      # 정수형 데이터 할당
name = "Alice" # 문자열 할당
```

2.3 데이터 입력과 출력

- 표준 입력 함수: `input()` 을 사용해 사용자 입력 받기.
- 표준 출력 함수: `print()` 로 화면에 출력.

```
age = input("나이를 입력하세요: ")
print(f"입력하신 나이는 {age}세입니다.")
```

2.4 연산자

- 산술 연산자: `+`, `-`, `*`, `/`, `//`, `%`, `**`

```
# 덧셈 예제
a = 5 + 3      # 8
# 뺄셈 예제
b = 10 - 4     # 6
# 곱셈 예제
c = 7 * 2      # 14
# 나눗셈 예제
d = 15 / 3     # 5.0 (실수형 결과)
# 몫 구하기 예제
e = 15 // 4    # 3
# 나머지 구하기 예제
f = 15 % 4     # 3
# 제곱 연산 예제
g = 2 ** 3     # 8
```

2.5 자료형 변환

- `int()`, `float()`, `str()` 등을 사용하여 자료형 변환 가능.

3. 조건문과 제어문

3.1 조건식

- 조건식: True 또는 False를 반환하는 식.

```
if score >= 60:    # if True이면 "합격"
    print("합격")
else:
    print("불합격")
```

3.2 비교, 논리 연산자

- 비교 연산자: `==`, `!=`, `>`, `<`, `>=`, `<=`

```
x = 5
y = 10
print(x == y) # False
print(x != y) # True
```

- 논리 연산자: `and`, `or`, `not`

```
a = True
b = False
print(a and b) # False
print(a or b)  # True
```

3.3 조건문 (if-elif-else)

- 여러 조건을 확인할 때 `if-elif-else` 를 사용.
- `elif`, `else`는 반드시 `if`와 함께 사용.

```
if score >= 90:
    print("A학점")
elif score >= 80:
    print("B학점")
else:
    print("C학점")
```

4. 반복문

4.1 for문

- **특징:** 반복 가능한 객체(리스트, 문자열 등)를 순회하며 루프 실행.
- **예제:**

```
fruits = ['apple', 'banana', 'cherry']

for fruit in fruits:
    print(fruit)
```

- **range() 함수 :** 연속된 숫자(정수)를 만들어 줌.
- **슬라이싱 활용:** 특정 범위 내 요소 반복.

4.2 while문

- **특징:** 조건이 참일 동안 루프 실행.
- **예제:**

```
count = 0
while count < 5:
    print(f"Count: {count}")
    count += 1
```

4.3 break와 continue

- **break:** 반복문을 즉시 종료.
- **continue:** 해당 반복을 건너뛰고 다음 반복으로 이동.

```
for i in range(10):
    if i == 5:
        break    # 5에서 종료
    elif i % 2 == 0:
        continue # 짝수일 때 건너뛰
    print(i)
```

4.4 이중 for문

- **특징:** 반복문 안에 또 다른 반복문 작성.
- **예제:**

```
for i in range(3):
    for j in range(2):
        print(f"i: {i}, j: {j}")
```

5. 자료구조: 리스트, 튜플, 딕셔너리

5.1 리스트

- **특징:** 여러 값을 하나의 변수에 저장 가능, 값 변경 가능.
- **예제:**

```
fruits = ['apple', 'banana', 'cherry']
```

- **메서드 활용:**
 - 추가: `append()`, `insert()`.
 - 삭제: `remove()`, `pop()`.
 - 슬라이싱: 특정 범위 내 요소

```
print(fruits[1:3]) # ['banana', 'cherry']
```

5.2 튜플

- **특징:** 리스트와 유사하지만 요소 변경(추가/수정/삭제) 불가능.
- **예제:**

```
coordinates = (10, 20)
```

- **활용:** 변경할 필요 없는 데이터 저장 시 유용.

5.3 딕셔너리

- **특징:** 키-값 쌍으로 데이터를 저장. 중복 키 사용 불가. 키는 수정 불가능한 자료형 사용.

- **예제:**

```
person = {'name': 'Alice', 'age': 25}
```

- **메서드 활용:**

- 추가 및 수정: `dict[key] = value`
- 값 가져오기: `get()`
- 삭제: `del()`

```
print(person.get('name'))    # 'Alice'
```